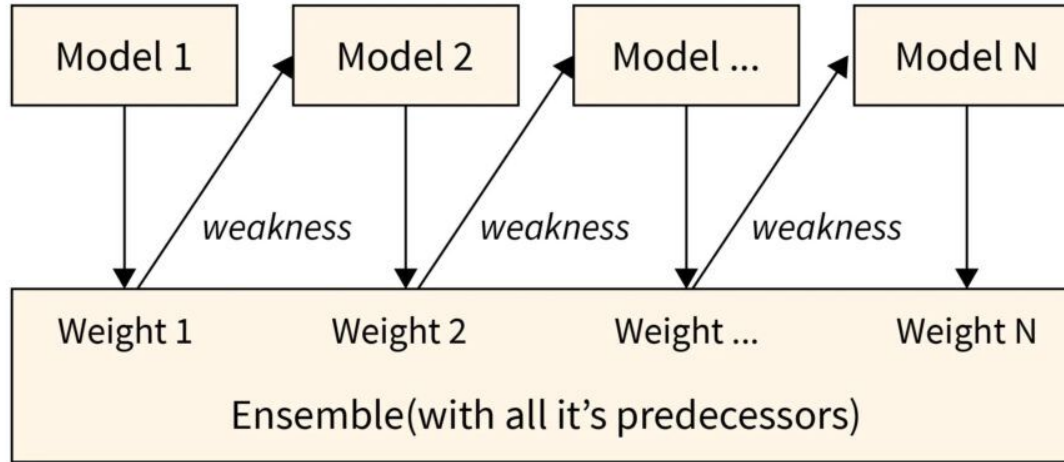


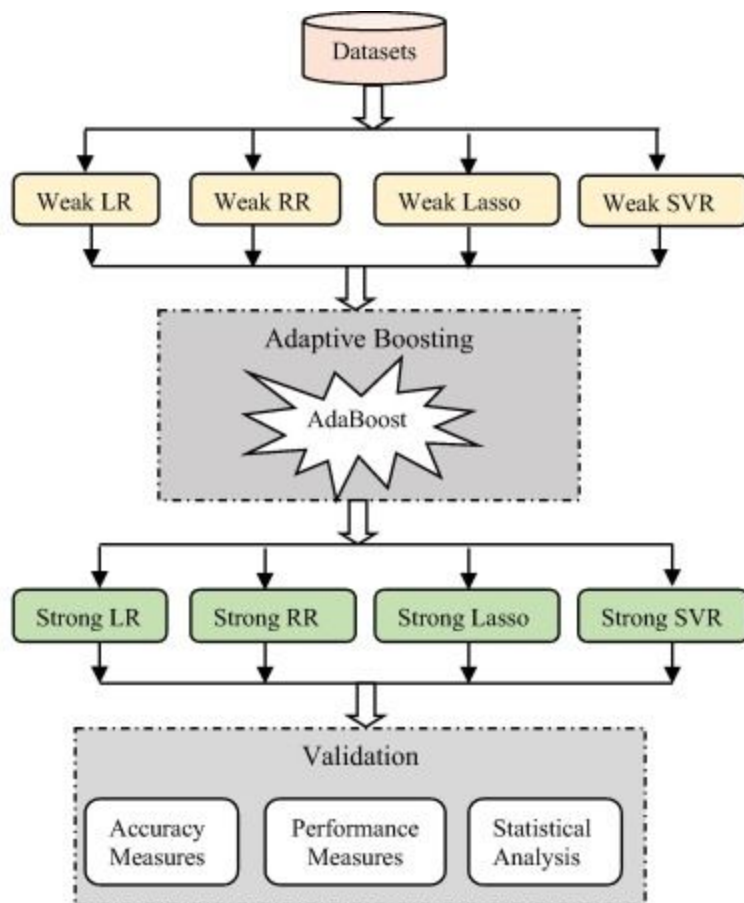
Boosting Algorithm

1. Ada boost

AdaBoost (Adaptive Boosting) is a supervised machine learning ensemble technique that combines multiple **"weak learners"** to create a single **"strong learner"**. It works sequentially, meaning each subsequent model aims to correct the errors of its predecessor.

Model 1,2,...,N are individual models (eg. decision tree)





How Ada Boos algor

1. **Train Weak Learner:** A simple model, most commonly a **decision stump** (a decision tree with only one split), is trained on the data.
2. **Calculate Error & "Amount of Say":** The algorithm calculates the weighted error rate of the stump. Based on this error, it assigns a weight (called **Alpha**) to that model; more accurate models get a higher "amount of say" in the final decision.
3. **Update Sample Weights:**
 - **Misclassified samples** receive higher weights so they are prioritized by the next learner.
 - **Correctly classified samples** receive lower weights.
4. **Repeat:** This process iterates for a specified number of learners (estimators) or until the training data is perfectly predicted.
5. **Final Prediction:** To make a prediction, AdaBoost performs a **weighted majority vote** across all weak learners. The learners with higher Alpha scores have more influence on the final result.

Characteristics of Ada Boost

- **Base Learners:** While any algorithm that accepts weighted training data can be used, **decision stumps** are the standard default.
- **Bias vs. Variance:** AdaBoost is primarily used to **reduce bias**.
- **Loss Function:** It typically minimizes an **exponential loss function**.
- **Hyperparameters:** Key parameters to tune include `n_estimators` (number of learners) and `learning_rate` (shrinks the contribution of each model)

2. XG Boost

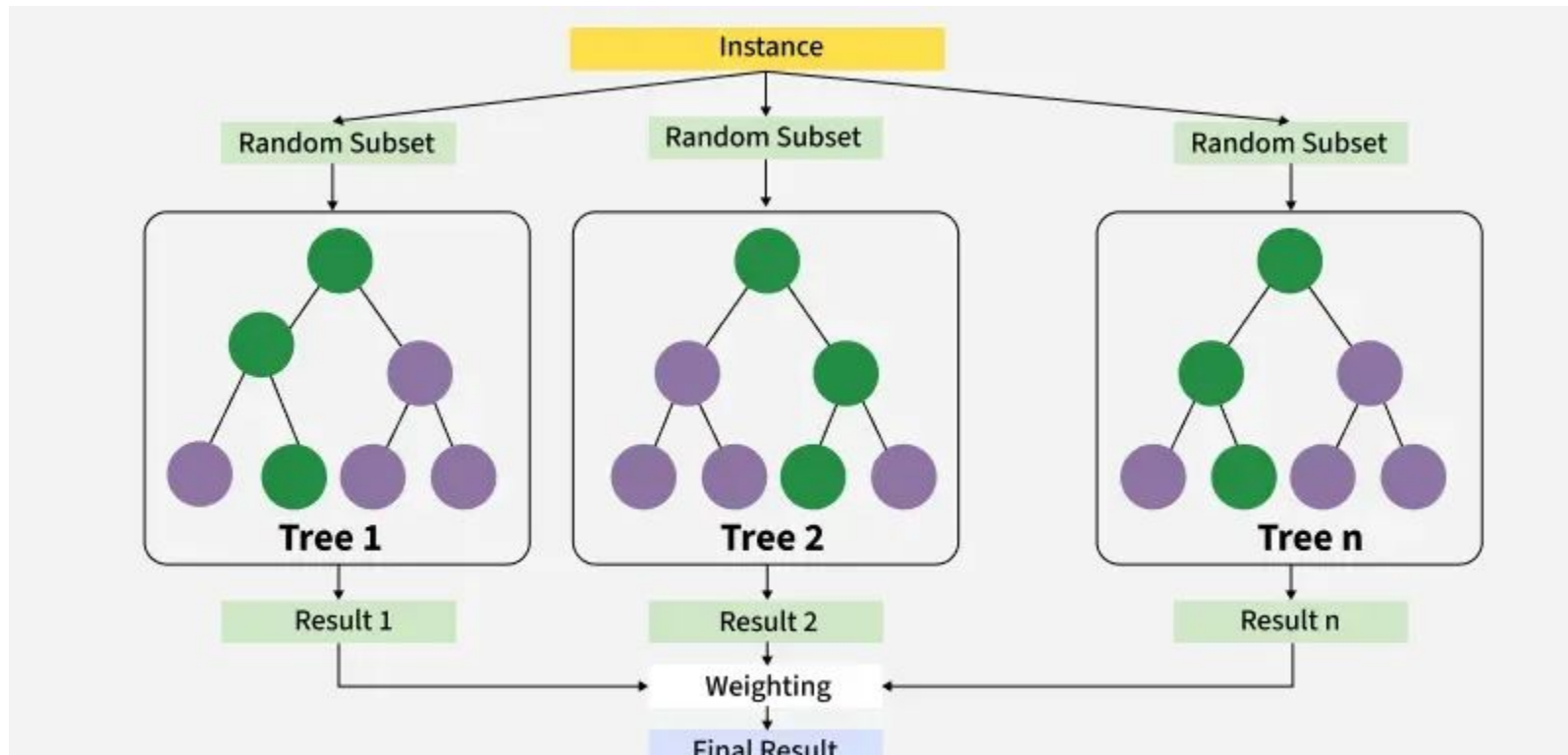
XGBoost (eXtreme Gradient Boosting) is highly efficient, popular, and powerful open-source machine learning library that implements gradient-boosted decision trees, excelling in classification, regression, and ranking tasks, especially with large, complex datasets due to its speed, scalability, and built-in optimizations like parallel processing and regularization.

...XG Boost - key features

It works by sequentially combining many simple "weak" decision trees, with each new tree focusing on correcting the errors of the previous ones, leading to a strong, accurate final model.

- Gradient Boosting: Builds models in stages, with each new tree correcting the mistakes (residuals) of the combined previous trees, explained [in this article on Analytics Vidhya](#) and [this video on YouTube](#).
- eXtreme Optimization: Highly optimized for speed and performance, using parallel processing and efficient tree-building algorithms.
- Regularization: Includes L1/L2 regularization to prevent overfitting, a common issue with complex models, making it robust.
- Missing Value Handling: Has built-in routines to handle missing data, reducing the need for extensive preprocessing, notes [an article on Medium](#).
- Scalability: Designed to handle large datasets efficiently, suitable for big data applications.

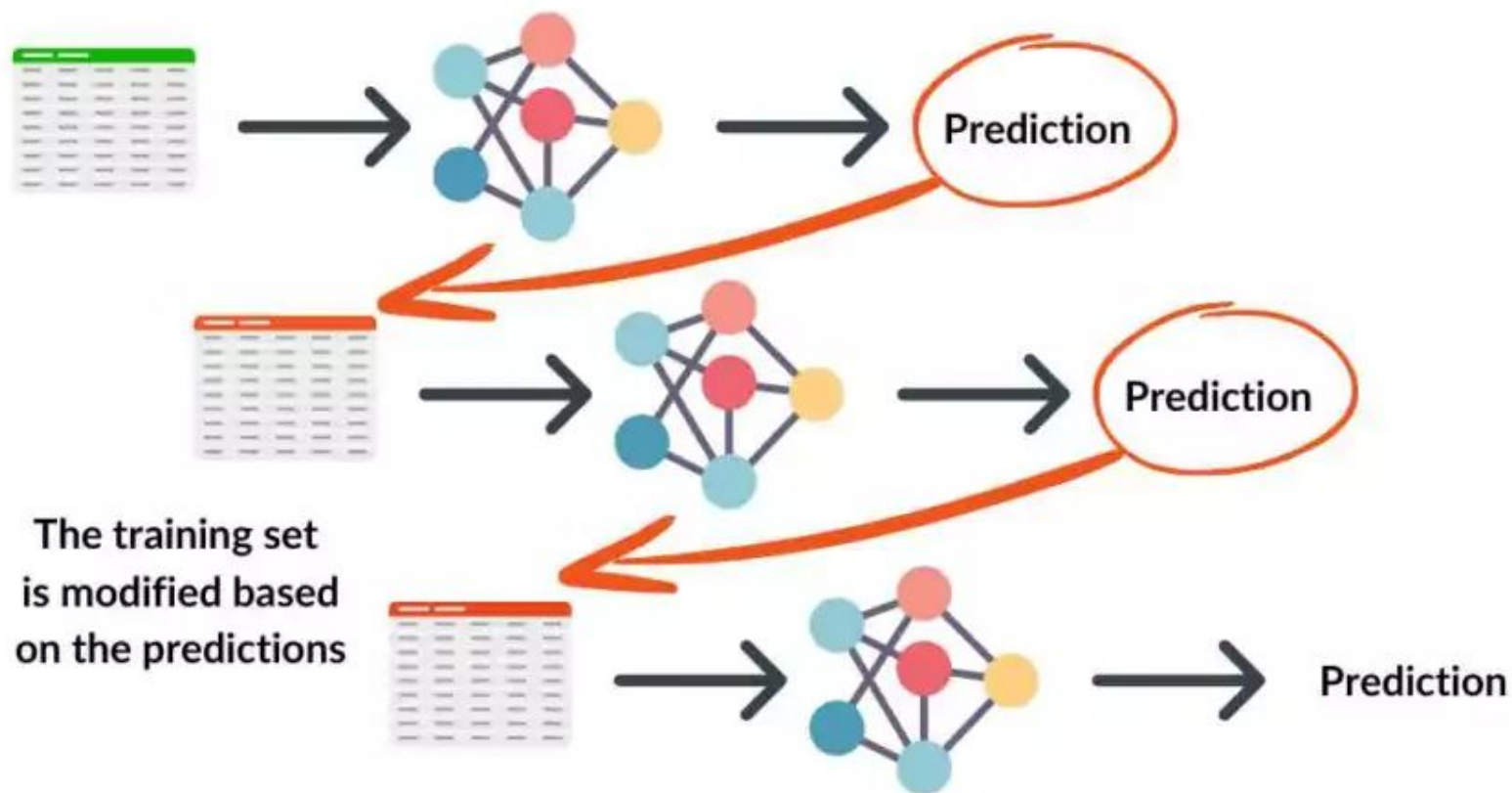
...XG Boost



...XG Boost



Base Models



LG Boost Algorithm

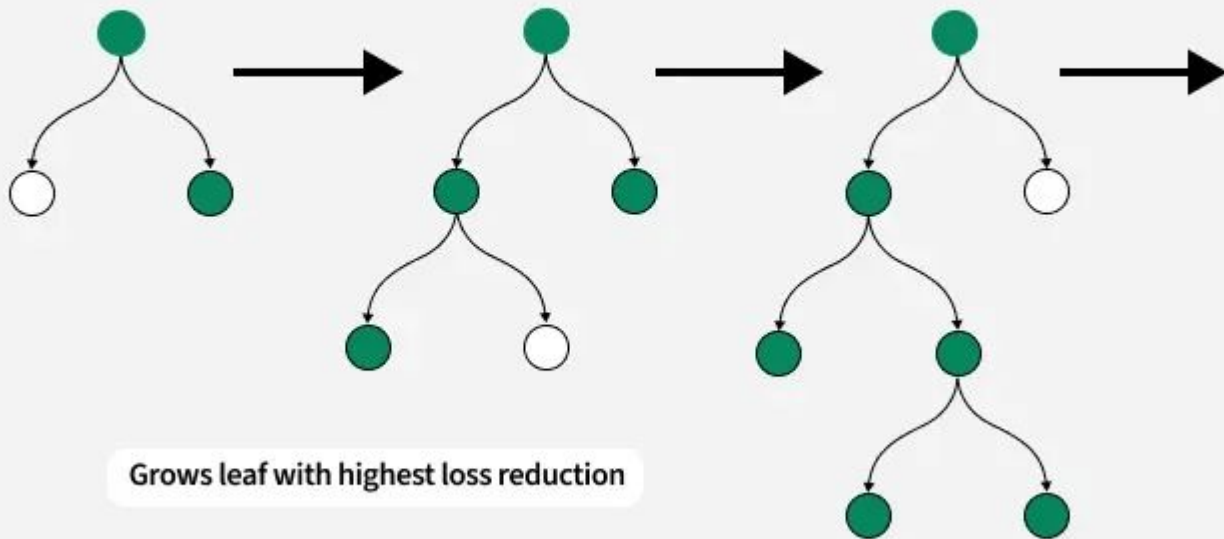
LightGBM is an open-source high-performance framework developed by Microsoft. It is an ensemble learning framework that uses gradient boosting method which constructs a strong learner by sequentially adding weak learners in a gradient descent manner.

...cont

It's designed for efficiency, scalability and high accuracy particularly with large datasets. It uses decision trees that grow efficiently by minimizing memory usage and optimizing training time. Key innovations like Gradient-based One-Side Sampling (GOSS), histogram-based algorithms and leaf-wise tree growth enable LightGBM to outperform other frameworks in both speed and accuracy.

...cont...

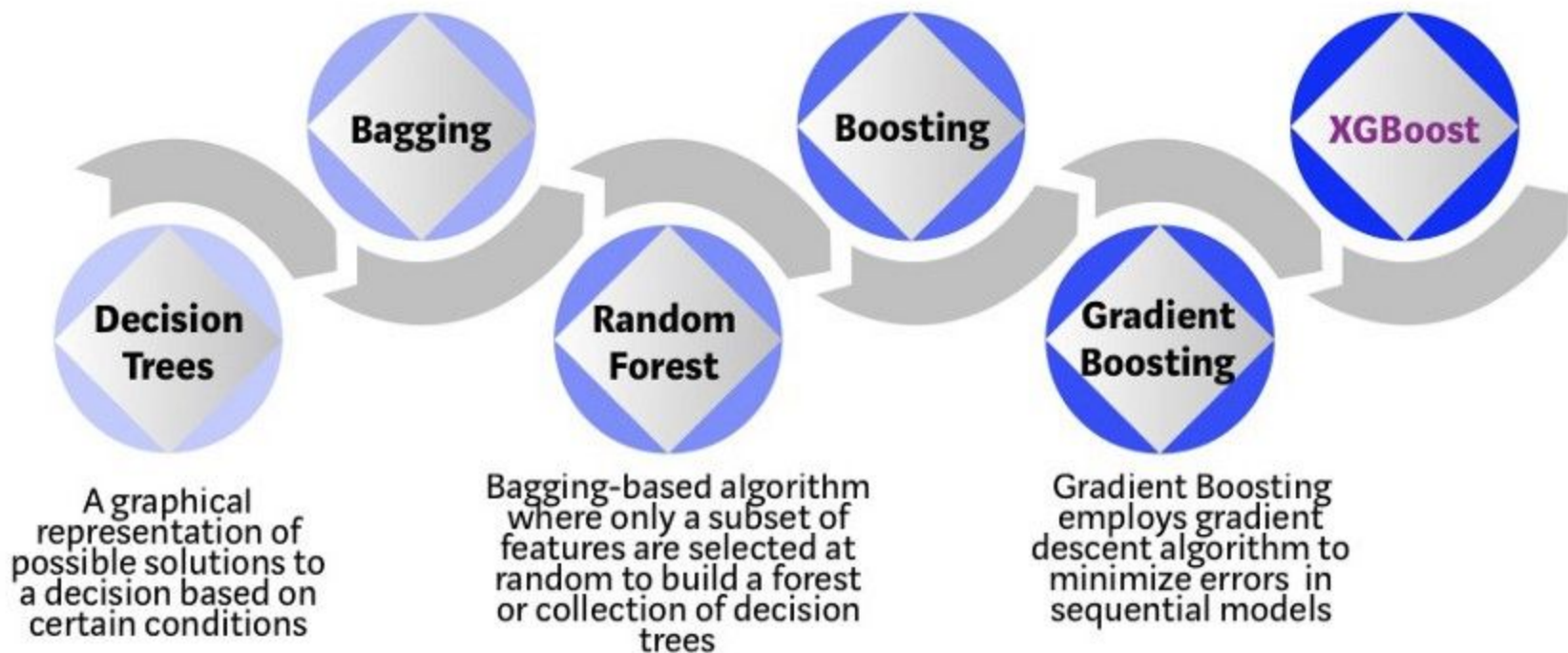
How LightGBM Works



Bootstrap aggregating or Bagging is a ensemble meta-algorithm combining predictions from multiple-decision trees through a majority voting mechanism

Models are built sequentially by minimizing the errors from previous models while increasing (or boosting) influence of high-performing models

Optimized Gradient Boosting algorithm through parallel processing, tree-pruning, handling missing values and regularization to avoid overfitting/bias



...cont

Tips: Using LightGBM Algorithm Effectively

Parameter Tuning

Feature Engineering

Early Stopping

Overfitting Prevention



END

&

Thank you